

Lezione 12 11/12/23 Eccezioni

giovedì 11 gennaio 2024 11:18

In un qualsiasi programma, oltre ad effettivamente le righe di codice, bisogna progettare anche la **logica di errore**: logiche di controllo per corretta gestione degli errori. Principalmente ci sono 3 tipi di errori: sintassi (fase di compilazione), comportamento (fase test ed unità) ed interazione (fase di test di integrazione). **Nota: un qualunque sistema non è privo di errori: non si ha la certezza di assenza di assenza in un qualunque sistema sw.** Per evitare ciò (crash del programma) si devono progettare queste logiche di controllo degli errori in modo parallelo alla progettazione del sistema (comportamenti desiderati). Andiamo a vedere come si progetta la logica di gestione: inizialmente si definivano delle convenzioni (valori di ritorno delle funzioni speciali, oppure si usava un registro / flag globali); mentre i moderni linguaggi di programmazione usano costrutti nativi per questa esplicita gestione: nel caso di UML si usano gli exit points, mentre nel caso di java si usano le **EXCEPTIONS**. Dopo questa breve intro, andiamo a vedere ora cosa sono effettivamente queste eccezioni: **per eccezione si intende un qualunque evento che si verifica durante esecuzione di azioni in logica di controllo, che ne interrompe il normale flusso, alterando così la sequenza definita dal programmatore.** Vediamo alcuni esempi di eccezioni: si rompe hd, si accede a valore fuori di array dichiarato, connessione diretta persa, (de)serializzazione di oggetto non va buon fine, oppure il file non ha sufficienti permessi. Vediamo un esempio:

```
readFile {  
    open(file);  
    s = size(file);  
    mem = allocate(s);  
    read(file, mem);  
    close(file);  
}
```

Questa funzione apre un file, ne verifica la dimensione e copia il contenuto del file in memoria, dopo aver verificato che vi sia sufficiente spazio. In linea di principio funzionerebbe, ma manca la logica di gestione dell'errore: ma che succede se il file non può essere aperto, la lunghezza del file non può essere determinata, non vi è abbastanza memoria libera da allocare, se lettura / chiusura fallisce?? Si entra in un comportamento indesiderato, il quale può portare ad inconsistenza dei dati.

Vediamo una possibile migliona:

```
readFile {  
    errcode = 0;  
    open(file);  
    if (!failed){  
        s = size(file);  
        if (s > 0){  
            mem = allocate(s);  
            if (isFree(mem)){  
                read(file, mem);  
                if (!failed)  
                    errcode = -1;  
            }else  
                errcode = -2;  
        }else  
            errcode = -3;  
        close(file);  
        if (closeFailed && errorCode == 0)  
            errorCode = -4;  
        else  
            if (errorCode != 0)  
                errorCode = -5;  
        }else  
            errorCode = -6;  
        return errorCode;  
    }  
}
```

Questo programma scritto così ancora non va bene, in quanto ancora non si riesce a capire cosa fa: in quanto la logica di errore e di controllo sono mischiate. Aumenta la complessità del codice e cosa succede se dovessi modificarlo?? Aumenta la difficoltà della gestione.

Quindi vediamo una soluzione più elegante:

```
readFile {  
    try{  
        open(file);  
        s = size(file);  
        mem = allocate(s);  
        read(file, mem);  
        close(file);  
    }catch (failedOpen){  
        ...  
    }catch (noSize){  
        ...  
    }catch (failedCopy){  
        ...  
    }catch (failedClose){  
        ...  
    }  
}
```

In questa soluzione più compatta le logiche di gestione e di controllo sono gestite separatamente.

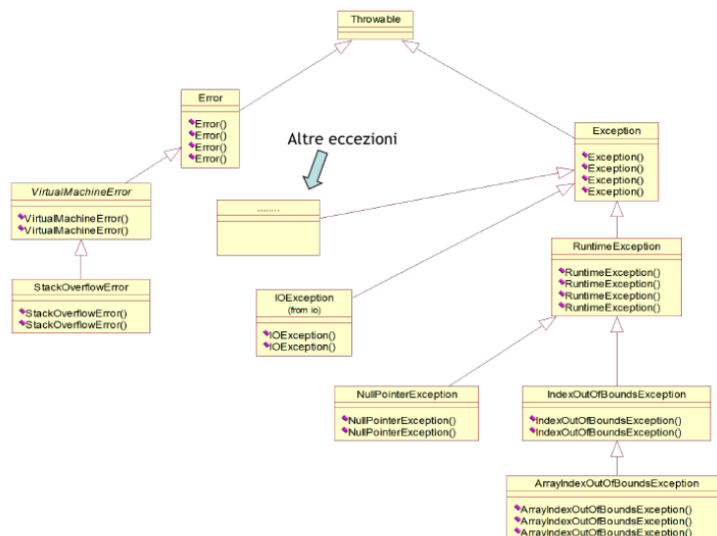
Andiamo ora vedere come effettivamente si intercettano le eccezioni: se in presenza di eccezione si cerca di porre rimedio:

```
try {  
    <IstruzioniLogicaDiControllo>  
}catch (<Tipo eccezione> <Identificatore>){  
    <IstruzioniLogicaErrore>  
}
```

Nel blocco **try** vi si definiscono delle istruzioni nelle quali vi si possono generare eccezioni , mentre nel blocco **catch** definiscono i blocchi effettivi nelle quali vengono gestite le eccezioni : possono essercene di vari . Nel caso che si entri nel catch , l'esecuzione del programma salta alla computazione della prima istruzione del blocco catch corrispondente, il quale match avviene in base al tipo (catena di generalizzazioni). Da notare che questo blocco può avere ulteriore clausola : il **finally** il quale blocco di codice viene sempre eseguito. In generale se logica di errore si eseguono i seguenti passi :

1. Si entra nel try
2. Se errore/eccezione si entra nel catch corrispondente
3. Se catch cattura eccezione , dopo la gestione si entra nel finally
 - a. Se catch non cattura eccezione , in quanto non c'è si va diretti nel finally.

Riepilogando : **le eccezioni possono essere racchiuse all'interno di un blocco try / catch , il quale blocco definisce opportuna logica di gestione dell'errore** : ma se non fossimo in grado di gestire opportunamente un errore?? si deve identificare il metodo (implementazione dell'operazione) come possibile sorgente di errore non gestito : il chiamante (programmatore) deve farsi carico di gestire in modo opportuno l'errore. In questo caso la dichiarazione del metodo usa una particolare clausola : **throws** , la quale permette al metodi di lanciare un'eccezione , la quale deve essere comunque gestita in modo opportuno : ad interno di try-catch. Vediamo ora come progettarle : come già detto questo step della progettazione di un qualunque sistema sw è importantissimo , ed anche qui è possibile applicare tecniche/strategie/pattern già visti, in quanto anche le eccezioni sono oggetti (class object). **Ricordiamo che anche per le eccezioni vale la gerarchia** . Ciò permette di gestire e/o definire una tassonomia di errori nei quali il sistema può incorrere : aumenta la semplificazione delle strategie di recupero e/o segnalazione in caso di errori in corso , quindi si arriva ad una semplificazione della manutenzione della gestione della logica di errore . In dettaglio :



Da notare che se si è in presenza di ERRORI (java.lang.error) non è possibile effettuare il recupero : interruzione della logica di controllo porta al crash del sistema; mentre per quanto riguarda le eccezioni (java.lang.exception) è possibile gestirle e recupero.

Attenzione però : le eccezioni sono di due tipi : unchecked e checked : le prime sono quelle di RuntimeException e sue sottoclassi , le quali non devono essere gestite esplicitamente dal programmatore , e principalmente si verificano quando è stato commesso un errore di programmazione : cast oppure accesso a puntatore nullo . Le altre invece necessitano di opportuna gestione(try/catch/throws) : si verificano quando è successo qualcosa di imprevisto. Le principali operazioni sono le seguenti :

- getMessage ()
 - accede al messaggio contenente i dettagli dell'eccezione
- toString ()
 - restituisce una breve descrizione dell'oggetto Throwable, compresi i dettagli dell'eccezione, se esistenti
- printStackTrace ()
 - stampa sullo standard di error l'eccezione con il relativo stack delle chiamate
- getCause ()
 - da Java 1.4 è possibile impostare l'eccezione originale come "causa" della nuova eccezione ovvero si possono annidare eccezioni
- initCause(Throwable cause)
 - configura/sovrascrive la causa di una eccezione

Come già detto : dato che le eccezioni sono figlie di classe object , possono essere soggette a conversioni :

1. Conversione esplicita
 - a. Questa tecnica permette di propagare errori agli starti esterni del sistema, mascherando così la vera causa dell'errore/eccezione.
 - b. Quindi si deve definire un vocabolario di errori comprensibili dal chiamante di una specifica API , con indipendenza dal modo di realizzazione
 - c. Gestione dell'eccezione interna creando e propagando nuova istanza di eccezione esterna comprensibile e gestibile dal chiamante
 - d. Vediamo un esempio : scrittura su file
 - i. Viene sollevata java.io.IOException

- ii. Quindi si deve catturare e gestire questa eccezione con il try/catch , e questa eccezione viene "convertita con una user-friendly" : magari con errore più semplice ed esplicativo : così si propaga nuova eccezione perdendo quella interna
 - iii. eccezione con messaggio di errore human-readable --> *"Spiacente, si è verificato un problema tecnico, riprovare in seguito"*
- 2. Exception chaining/wrapping
 - a. Con questa tecnica si propaga/comunica errore a strato sw più esterno , mascherando il vero motivo del problema , **ma si mantiene traccia del perché vi è stato errore/eccezione.**
 - b. Quindi si deve specificare un vocabolario comprensibile da utente con indipendenza dal modo di realizzazione
 - c. Gestione di istanza interna e propagazione/gestione di quella esterna
 - d. Documentare eccezione esterna aggregando istanza interna tra le cause del problema
 - e. Vediamo un esempio : scrittura su file
 - i. Viene sollevata java.io.IOException
 - ii. Quindi si deve catturare e gestire questa eccezione con il try/catch , e questa eccezione viene "convertita con una user-friendly" : magari con errore più semplice ed esplicativo : così si propaga nuova eccezione perdendo quella interna
 - iii. eccezione con messaggio di errore human-readable --> *"Spiacente, si è verificato un problema tecnico, riprovare in seguito"*
 - iv. Quindi si propaga eccezione **mantenendo** traccia dell'errore interno : dettaglio errore mantenuto in un log
- 3. Overriding
 - a. Se in questo caso, le eccezioni previste hanno impatto anche sui metodi ù
 - b. Più in dettaglio :
 - i. In sottoclasse possono essere gestite/lanciate solo quelle lanciate dal metodo/operazione della classe base, avendo così la sicurezza che il codice funzioni con la classe base , così per catena anche con tutte le sottoclassi
 - ii. **Analogamente se si ha a che fare con interfacce**
 - iii. Invece per quanto riguarda i costruttori non vale questa proprietà : i costruttori di una sottoclasse possono lanciare una qualunque eccezione : attenzione però : le eccezioni della superclasse devono essere gestite correttamente nella sottoclasse , in quanto il costruttore di una sottoclasse come prima operazione invoca il costruttore della superclasse (super).